Práctica 0 · Auditoría de nuestro entorno de trabajo

Sistemas Informáticos · 1º DAM

Audita **tu propio equipo** como si fueras el técnico que lo recibe: qué tiene, cómo rinde y cómo mejorarlo. Entregas un **informe** en `auditoria_apellidoNombreAlumno.md`, subido a GitHub; las capturas van en una carpeta `capturas/` del mismo repositorio y se enlazan desde el informe.

Fases del trabajo

Fase	Qué tienes que entregar	Peso
1 · El origen de la máquina	¿Cuándo compraste el equipo y con qué presupuesto? ¿Por qué lo compraste y qué buscabas (trabajo, estudios, gaming, movilidad…)? ¿Cuál fue tu prioridad principal?	10 %
2 · La autopsia del hardware	**CPU:** modelo exacto, generación, núcleos, velocidad. **RAM:** capacidad, velocidad, tipo (DDR4/DDR5), zócalos totales y ocupados. **Almacenamiento:** capacidad, tipo (HDD, SSD SATA, SSD NVMe M.2), particiones. **GPU:** integrada o dedicada, VRAM. **Evidencia:** la salida de cada comando (sección «Fase 2 · Matriz de herramientas»).	30 %
3 · Prueba de rendimiento	Con el IDE, el navegador y una base de datos abiertos a la vez, vigila el **Administrador de tareas** (Windows) o el **Monitor del sistema** (Linux): ¿qué colapsa primero: la CPU (100 %), la RAM (swap / archivo de paginación) o el disco (tiempos de respuesta altos)? Cruza con las fases 1 y 2: ¿fue una compra acertada? Con capturas.	30 %
3 · Plan de mejora	¿Tu equipo se puede ampliar o tiene componentes soldados a la placa base? **Sí:** ¿qué cambiarías y por qué? **No:** ¿qué especificaciones mínimas innegociables buscarás en tu próxima compra?	20 %
4 · La documentación es código	**Documentación:** informe en **Markdown** (no Word) bien estructurado y entregado vía **GitHub**. Usa `#` `##` `###` para estructurar las fases; listas, **negritas** y tablas para el resumen del hardware; y bloques de código (tres acentos graves) para la salida de los comandos.	10 %

Flujo de entrega en GitHub

Antes del paso 1 necesitas una **cuenta de GitHub** (con el correo verificado), **Git instalado** (compruébalo con `git --version`) y la **terminal abierta en la carpeta del proyecto**.

Crea el **repositorio** en GitHub sin README: puede ser único para todas las prácticas (una carpeta por práctica) o uno por práctica; te aconsejo **privado**. **Compártelo conmigo**: *Settings* → *Collaborators* → *Add people* e invita a `ivan.saugar@retamar.es` (si da fallo, a `isaugar`).

> **La primera vez que uses Git** (una sola vez por ordenador), antes del paso 1:
> `git config --global user.name "Tu Nombre"`
> `git config --global user.email "tu@correo"`

Paso 1 · Inicializar	Paso 2 · Preparar	Paso 3 · Sellar	Paso 4 · Desplegar
`git init -b main` `git remote add origin <URL>`	`git status` `git add .`	`git commit -m "docs: auditoria del equipo"`	`git push -u origin main`
Crea el repositorio local y conéctalo con el de GitHub.	Añade el `.md` y las capturas al área de preparación.	Registra los cambios de forma permanente.	Sube el código al repositorio remoto.

Ojo: en el primer `push` se abrirá una ventana para iniciar sesión en GitHub.

> ⚠️ **Ten cuidado al añadir información personal** en el informe, las salidas y las capturas.

Fase 2 · Matriz de herramientas de extracción: la terminal

Usa la **terminal de tu sistema real**: **PowerShell** en Windows o **Bash** en Linux (`dmidecode` necesita `sudo`). Los comandos deben mostrar **tu hardware real**, no uno virtual: ni una máquina virtual ni WSL sirven (WSL no muestra bien la RAM, los discos ni los zócalos). La evidencia es la **salida de cada comando** (captura de la terminal); CPU-Z, HWiNFO64, CrystalDiskInfo (Windows) o CPU-X, Hardinfo (Linux) solo contrastan y **no sustituyen** a los comandos. Antes de ejecutar cada comando, revisa qué hace.

Windows · PowerShell

```
# CPU: modelo, núcleos, hilos y frecuencia base (MHz)
Get-CimInstance Win32_Processor | Format-List Name, NumberOfCores, NumberOfLogicalProcessors,
MaxClockSpeed
# RAM: un módulo por fila (Capacity en bytes, Speed en MT/s, tipo: 26 = DDR4, 34 = DDR5)
Get-CimInstance Win32_PhysicalMemory | Format-Table DeviceLocator, Capacity, Speed, SMBIOSMemoryType -
AutoSize
# RAM: zócalos totales (los ocupados son las filas del comando anterior)
Get-CimInstance Win32_PhysicalMemoryArray | Format-List MemoryDevices
# Discos: tipo (SSD/HDD), bus (NVMe/SATA), tamaño en bytes, esquema (GPT/MBR) y particiones
Get-PhysicalDisk | Format-Table FriendlyName, MediaType, BusType, Size -AutoSize
Get-Disk | Format-Table Number, FriendlyName, PartitionStyle, Size -AutoSize
Get-Partition | Format-Table DiskNumber, PartitionNumber, DriveLetter, Type, Size -AutoSize
# GPU: modelo y driver
Get-CimInstance Win32_VideoController | Format-List Name, DriverVersion
# VRAM de una NVIDIA (no uses AdapterRAM: Windows lo corta en 4 GB)
nvidia-smi --query-gpu=name,memory.total --format=csv
# VRAM de cualquier GPU (el fichero tarda ~30 s en aparecer)
dxdiag /t "$env:TEMP\dxdiag.txt"
Select-String -Path "$env:TEMP\dxdiag.txt" -Pattern "Card name|Dedicated Memory|Shared Memory"
# Virtualización (True = activada)
(Get-CimInstance Win32_Processor).VirtualizationFirmwareEnabled
```

Linux · Bash

```
# CPU: modelo, CPU(s) = hilos, núcleos por socket y frecuencia máxima
lscpu
# RAM: total, usada y swap
free -h
# RAM: módulos y zócalos («Size: No Module Installed» = zócalo libre)
sudo dmidecode -t memory | grep -E "Number Of Devices|Locator|Size:|Type:|Speed:"
# Discos: ROTA 1 = HDD, 0 = SSD; bus (sata/nvme); esquema de particiones (gpt/dos)
lsblk -d -o NAME,SIZE,ROTA,TRAN,PTTYPE,MODEL
# Particiones: tamaño, sistema de archivos y punto de montaje
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT
# GPU: modelo
lspci | grep -iE "vga|3d|display"
# VRAM de una NVIDIA (con su driver propietario)
nvidia-smi --query-gpu=name,memory.total --format=csv
# VRAM de una AMD (en bytes)
cat /sys/class/drm/card*/device/mem_info_vram_total
# Virtualización (debe aparecer VT-x o AMD-V)
lscpu | grep -i virtualiz
```

Cómo leer la salida

- **Tamaños:** Windows da `Capacity` y `Size` en **bytes**. Para pasarlos a **GB** (decimales) divide entre 1.000.000.000; para pasarlos a **GiB** (binarios) divide entre 1.073.741.824. Anota cuál usa cada fuente (el fabricante anuncia en GB; el sistema suele mostrar GiB).
- **CPU:** `MaxClockSpeed` es la frecuencia **base**; en Linux, «CPU max MHz» suele incluir el turbo. La generación se busca en la ficha del fabricante con el **modelo exacto**.
- **Gráfica integrada:** no tiene VRAM propia, usa RAM compartida («Shared Memory» en `dxdiag`).

Checkpoint crítico: virtualización

En esta clase usaremos máquinas virtuales y contenedores, así que la **virtualización por hardware** debe estar activada: **VT-x** (Intel) o **AMD-V** (AMD; en la BIOS suele llamarse *SVM Mode*). Compruébalo con el último comando de cada bloque; si no está activada, se activa en la BIOS/UEFI.